



CARE Knowledge Base

Knowledge Engine for CARE, by CARE!

Search by Category

All categories ▼ Search

Enter a search term, select a category and click **Search** or select an icon below.

[All Articles](#) [G2](#) [G3](#) [Car Park RMS](#) [RevPlan](#) [SmartSpace](#) [Casper](#) [Optix](#) [Assets](#)

Atlantis Bahamas Integration Landscape and guidance for support page

Summary

Atlantis Integration Flow

Atlantis Integration follows event-driven pattern: that is implemented by EventBridge, SQS and Lambda on AWS point of view and Spring Integration + Spring AMQP from application point of view

AWS Architecture best describes the following diagram:

1. Client uploads a file in S3 bucket backed by AWS transfer through SFTP protocol
2. Once file is uploaded, S3 generates S3ObjectCreated event and sends it to EventBridge in default bus
3. Based on rules, events are routed to multiple destinations: trigger relay Lambda function and log events in datadog
4. Relay Lambda serves as an inbound VPC Gateway, allowing event bridge send events in private destinations
5. In case of Atlantis App in unavailable for some reason (deployments, containers in ECS clusters are not running, etc.) events will be put in a Dead Letter Queue in SQS service so once Atlantis is up and running these events will be picked by app anyways
6. EventBridge provides and ability to replay all the event for specified time frame – Archive – so we can perform regression testing or at least verify existing functionalities after some code changes
7. Once Atlantis consumed event, it pulls newly created file from S3 and processes it: transform and send to pms-inbound through Rabbit Message Broker

There are 2 versions of it

The simplified version as follows

Categories

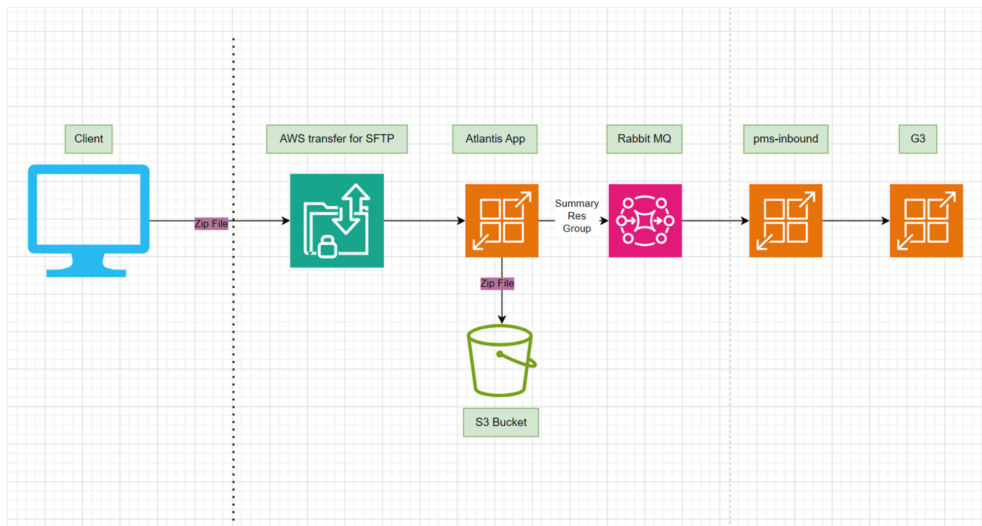
Select Category

Recent Posts

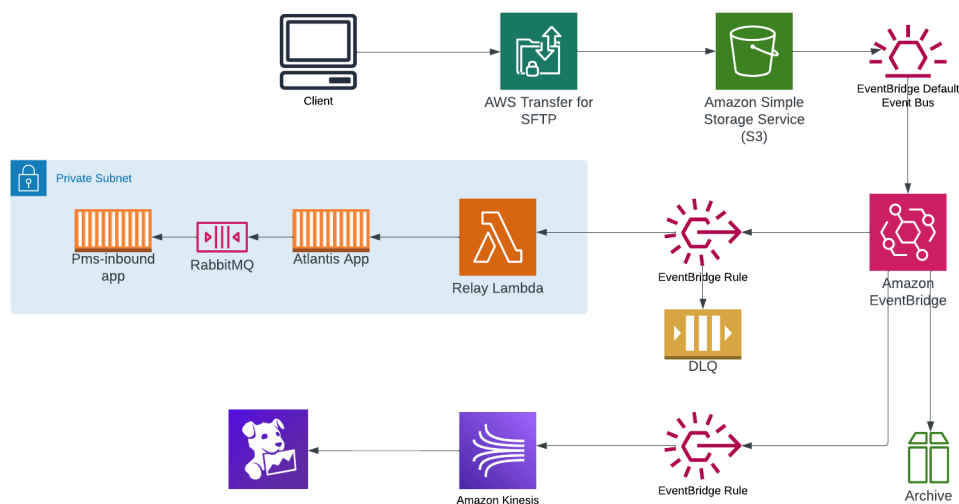
- [HLTN/hiltonEnableSmallGroupProductsAndInventoryLimitForNewPropertyStep](#)
- [Service Recovery Procedure – IDEaSOperaAgent Service in “Stopping” State](#)
- [Processing Failures Due to Data Type Mismatch for MGM Resorts](#)
- [Dynamic Optimization](#)
- [HLTN/Handling Error for SAS Dataset Encoding Job](#)

Recent Comments

- Pranav Thakar on [Hilton/Processing Failure – non positive offset value for Product](#)
- Ranjit on [Hilton/Processing Failure – non positive offset value for Product](#)
- Ranjit on [Service Recovery Procedure – IDEaSOperaAgent Service in “Stopping” State](#)
- Ranjit on [Processing Failures Due to Data Type Mismatch for MGM Resorts](#)
- Ranjit on [Accor Client Handling Process Guide](#)



Technically a more accurate view of what truly occurs is as follows



This document outlines the file specifications for inbound data in the context of IDeaS. These specifications are designed to facilitate the collection of data for the Atlantis Client. However, it's important to note that the file formats are generic and can be adapted to accommodate any client that provides data in PSV (Pipe-Separated Values) files.

Please find below overview for IDeaS File Format integration:

Data Integration Process for IDeaS Service

1. File Preparation:

- The client is responsible for creating a **ZIP file** according to the specified format.
- This ZIP file should contain the necessary data files.
- The client then pushes this ZIP file to the designated **SFTP location**.

2. Data Transformation:

- The IDeaS service monitors the SFTP location.
- When the ZIP file is detected, the service retrieves it.
- The data within the ZIP file is transformed into the **pms-inbound format**.

3. Populating Data in G3:

- Once the data is successfully transformed into pms-inbound format, the subsequent steps mirror the process used for other integrations (such as HTNG, OXI, and OHIP).
- The transformed data is loaded into the **G3 system** for further processing and analysis.

The IDeaS file integration accommodates both the **Summary Flow** and the **Inventory Flow**. In either case, the IDeaS integration process anticipates a **ZIP file** containing a set of PSV (Pipe-Separated Values) files. These files include data related to:

1. **Reservations**
2. **Groups**
3. **Inventory/Summary**

By adhering to this format, clients can seamlessly integrate their data with IDeaS, ensuring efficient and accurate information flow.

- **Summary Flow:**
 - Zip file format:
 - File Name: ClientCode_PropertyCode_YYYY-MM-DDTHHMMSS
 - Zip File: [CLIENTCODE_PROPERTYCODE_2023-08-22T094400.zip](#)
 - Zip file contents:
 - Reservations
 - Groups
 - Hotel Summary
 - Room Type Summary
 - Room Type Market Segment Summary
 - Inventory Count by Room Number (for Component room properties)
- **Inventory Flow:**
 - Zip file format:
 - File Name: ClientCode_PropertyCode_YYYY-MM-DDTHHMMSS
 - Zip File: [CLIENTCODE_PROPERTYCODE_2023-08-22T094400.zip](#)
 - Zip file contents:
 - Reservations
 - Groups
 - Inventory
 - Inventory Count by Room Number (for Component room properties)

More details:

- EventBridge – https://aws.amazon.com/eventbridge/?nc1=h_ls
- S3 – <https://aws.amazon.com/s3/>
- Transfer Family – <https://aws.amazon.com/aws-transfer-family/>
- SQS – <https://aws.amazon.com/sqs/>
- Lambda – <https://aws.amazon.com/lambda/>

How it works

Pull mechanism periodically (check `atlantis.pullsftp.scheduling.cron` configuration) goes to sftp server (check `atlantis.pullsftp.servers` configuration) and checks for files, which were never processed yet by any node.

Then it creates track entry (PullZipFileFromSftpTrackEntry) for each zip file and pushes it to rabbit mq for work distribution between nodes.

Then each node picks up entry, goes to sftp server and transfers zip file to S3 bucket. Then it starts processing of the zip file. Check `atlantis.pullsftp.storage.bucketName` for details. Information about file downloading is stored on PullZipFileFromSftpTrackEntry level.

There is a handler for each known psv file type. This handler knows how to parse the file.

We use ETL approach here.

So, first we parse the file (Extract).

Then we do conversions and adjustments as needed according to our business logic (Transform).

And then we push things to pmsInbound (Load).

Atlantis Inbound part 1 & Atlantis Inbound Part 2

Also log expressions that can be helpful while troubleshooting:

Find file processing

```
env:dev service:atlantis ("starting processing zip file" OR "successfully processed zip file" OR "failed to process zip file")
```

Key logs inside the file processing plus errors in atlantis OR pmsInbound

```
env:dev AND (service:atlantis AND (@logger.name:pull-sftp OR ("Finished sending" AND "to pmsinbound") OR ("Finished getRmsExtract" OR "Failed getRmsExtract" OR "unable to mark nucleusStats"))) OR (status:error AND service:(atlantis OR "/ecs/dev/pmsinbound/")))
```

Logs in pmsInbound

```
env:dev service:"/ecs/dev/pmsinbound/" (@logger.name:org.springframework.data.mongodb.core.MongoTemplate OR "preSend on channel")
```

A Note on Component Rooms

The Atlantis EDW is unable to accommodate the component room reservations as required by IDeaS. Therefore, IDeaS internal integration service (Atlantis service) will need to build the custom logic to handle the component rooms reservations.

There are two data types where we need to handle the component rooms – [Reservations](#) and [Inventories](#).

Reservation

EDW provides the number of rooms in the Reservation Data. It has been agreed with Atlantis team that 99% of reservations where number of rooms are greater than 1 are component room types. So, we can build a logic to combine all the reservations where occupancy dates are duplicated.

In the below reservation data example (Table 1.1), it is observed that reservation ID 100 is duplicated for the same occupancy date, albeit with a different room type. Additionally, the Number_Of_Rooms is greater than 1.

In order to resolve below reservation, we will store the mapping of Component Rooms in Atlantis service (Table 1.2).

Based on the mapping table, Atlantis service will combine below 2 reservation and create one single reservation for a component room type (Table 1.3).

Reservation Data (Table 1.1)

Reservation_ID	Occupancy_Dt	Arrival_Dt	Departure_dt	Market_Code	Room_Type_Code	Rate_Value	Room_Revenue	Number_Of_R
100	9/1/2024	9/1/2024	9/2/2024	BAR	CTRS	200	230	2
100	9/1/2024	9/1/2024	9/2/2024	BAR	CTPK	150	180	2

Component Room type mapping table (Table 1.2)

ComponentRoomType	PhysicalRoomType
-------------------	------------------

CTRSPK	CTRS
CTRSPK	CTPK

Resolved Reservation (Table 1.3)

Reservation_ID	Occupancy_Dt	Arrival_Dt	Departure_dt	Market_Code	Room_Type_Code	Rate_Value	Room_Revenue	Number_Of_R
100	9/1/2024	9/1/2024	9/2/2024	BAR	CTRSPK	350	410	1

Inventories

EDW is not able to provide us Inventory data for Component room types. However, G3 requires the inventory data to be provided for Component room types. Therefore, we need to **build** the inventory data from reservations. This logic needs to be built in pms-inbound as pms-inbound has all the required information to **build** the summary from reservations data at Hotel and Room Type level.

Rooms Sold = Reconciled reservations at occupancy date and room type level.

Capacity = 0

Out Of Order = 0 (G3 has a configuration screen to provide OOO information for **component room types**.)

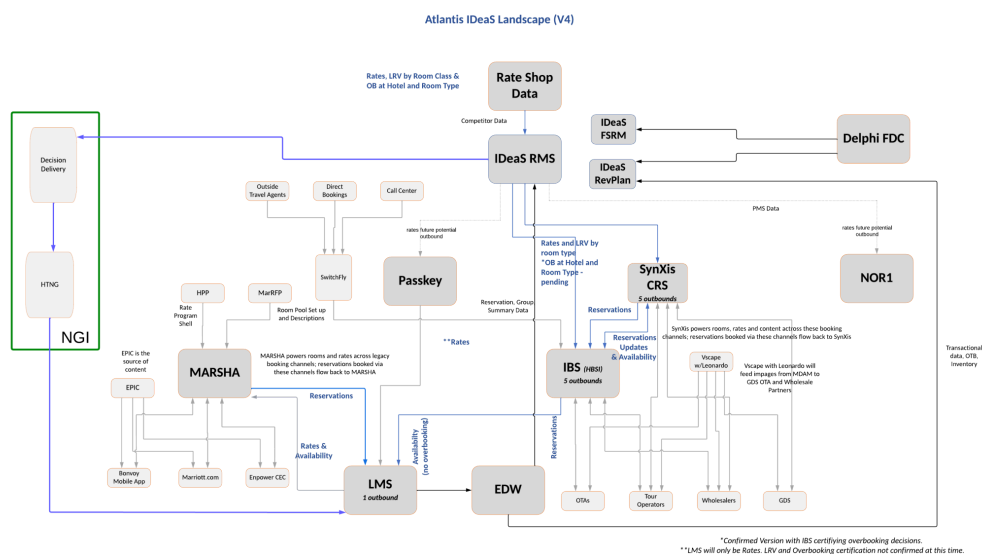
We might already have this functionality to build the summary data for component room types. (check resetVirtualCounts functionality)

Decision Upload Overview

Atlantis (BAM) is a complex implementation with a custom integration that kicked off in November of 2023. Conditional certification was approved in December 2024 and from January – April 2025 the Atlantis internal team has worked with the Atlantis external team to finalize certification to EDW/Snowflake, work through data validation, and implemented various business practice changes with the help of Solutions and Consulting IDEaS departments. Go live was approved on 11 April 2025 by the Atlantis team and is live as of 22 April 2025.

Integration Landscape

Below is the most current integration landscape as of 18 April 2025. For Phase 2 IDeaS G3 will upload to 11 outbounds which is detailed in the next section.



Upload Process and Decision Flow

Refer the below chart to understand which outbound is mapped to which vendor when you send emails for failures .

Atlantis System	Outbound G3 Name in Jobs	Tower Name	Vendor Mapping	Property Code	<u>Decisions Sent</u>
LMS	Decision Delivery1	All	Yes	0001	Daily BAR
IBS	HBSi	Coral Tower	Yes	937936	Daily BAR/LRV
IBS	SynXis6	Royal	Yes	937941	Daily BAR/LRV
IBS	SynXis7	Reef	Yes	937944	Daily BAR/LRV
IBS	SynXis8	Cove	Yes	937943	Daily BAR/LRV
IBS	SynXis9	Harborside	Yes	937942	Daily BAR/LRV
SynXis	SynXis	Harborside	Yes	11738	Daily BAR/LRV/OB
SynXis	SynXis1	Cove	Yes	17587	Daily BAR/LRV/OB
SynXis	SynXis2	Reef	Yes	19571	Daily BAR/LRV/OB
SynXis	SynXis3	Royal	Yes	11737	Daily BAR/LRV/OB
SynXis	SynXis4	Coral	Yes	63679	Daily BAR/LRV/OB

NOTICE if the outbound name has a 9 it would signal that it is for HBSi and if it does NOT begin with 9 then its Synxis.

Outbound Details: [ATL_Outbound Details](#)

Final Room Type Vendor Mapping: [Atlantis Tower Room Type Vendor Mapping](#)

Following is what the daily Decision upload flow looks like.

The 3 Decision Delivery1 jobs seen are for ONE outbound which in this case is LMS. Currently as of 25th Mar 2025 – the Base URL is set to redirected decision from G3>NGI Stage server>LMS Prod URL.

Very Important :

Notice HBSi , Synxis6,7,8,9 are for HBSi and NOT for Synxis (as outlined in the table above).
Synxis outbounds are Synxis , Synxis 1,2,3,4 . There is NO Synxis 5.

When issuing Monitoring failure emails please refer this document to better understand the vendor that you need to reach out to. If you do NOT understand – please ping Erin Voll/Yogesh Shinde and in their absence your respective managers for information.

Here's a sample of the investigation sample document created by Erin Voll for Atlantis based on configurations for the property.

[Atlantis Room Type Pricing Workbook](#)

Useful links

DataDog

- Atlantis app – [Atlantis V2](#)
- Relay Lambda – [Atlantis Lambda](#)
- EventBridge events – [Atlantis EventBridge](#)
- Decision Delivery : [Internal Link](#)

List of issues encountered :

12th May 2025 – Processing Halted. Error observed was as follows :

=====

javax.persistence.PersistenceException: org.hibernate.HibernateException: Transaction was rolled back in a different thread! Cause : org.hibernate.HibernateException: Transaction was rolled back in a different thread!

=====

DataDog error looked like below

=====

Encountered an error executing step: componentRoomRevenueSplitStep –
javax.persistence.TransactionRequiredException: no transaction is in progress
Work Context Info:
BkCorrelationId: null
ClientCode: ATLANTIS
ClientId: 674
ClientUpsId: null
ExternalSystem: null
FdId: null
PropertyCode: 0001
PropertyGroupId: null
PropertyId: 994565
PropertySFDCAccountNumber: null
PropertyStage: null
UserId: 11403
UserRole: BusinessUser
Uspld: 5a14f37f-f858-4ac1-81a8-2210770605e0
javax.persistence.TransactionRequiredException: no transaction is in progress
at
org.hibernate@5.3.10.Final//org.hibernate.internal.AbstractSharedSessionContract.checkTransactionNeede
dForUpdateOperation(AbstractSharedSessionContract.java:398)
at
org.hibernate@5.3.10.Final//org.hibernate.internal.SessionImpl.checkTransactionNeededForUpdateOperati
on(SessionImpl.java:3558)
at org.hibernate@5.3.10.Final//org.hibernate.internal.SessionImpl.doFlush(SessionImpl.java:1444)
at org.hibernate@5.3.10.Final//org.hibernate.internal.SessionImpl.flush(SessionImpl.java:1440)
at
org.jboss.as.jpa@17.0.1.Final//org.jboss.as.jpa.container.AbstractEntityManager.flush(AbstractEntityManage
r.java:459)
at deployment.platform-
ear.ear//com.ideas.tetris.platform.common.crudservice.CrudServiceBean.flush(CrudServiceBean.java:540)
at deployment.platform-
ear.ear//com.ideas.tetris.platform.common.crudservice.CrudServiceBean.flushAndClear(CrudServiceBean.ja
va:550)
at deployment.job-
web.war//com.ideas.tetris.platform.common.crudservice.CrudServiceBean\$\$FastClassBySpringCGLIB\$\$e5
2ddb2.invoke(<generated>)
at deployment.platform-
ear.ear//org.springframework.cglib.proxy.MethodProxy.invoke(MethodProxy.java:218)
at deployment.platform-
ear.ear//org.springframework.aop.framework.CglibAopProxy\$CglibMethodInvocation.invokeJoinpoint(Cglib
AopProxy.java:783)
at deployment.platform-
ear.ear//org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvoc
ation.java:163)
at deployment.platform-
ear.ear//org.springframework.aop.framework.CglibAopProxy\$CglibMethodInvocation.proceed(CglibAopPro
xy.java:753)
at deployment.platform-
ear.ear//org.springframework.transaction.interceptor.TransactionInterceptor\$1.proceedWithInvocation(Tra
nsactionInterceptor.java:123)
at deployment.platform-
ear.ear//org.springframework.transaction.interceptor.TransactionAspectSupport.invokeWithinTransaction(T
ransactionAspectSupport.java:388)
at deployment.platform-
ear.ear//org.springframework.transaction.interceptor.TransactionInterceptor.invoke(TransactionInterceptor.
java:119)

at deployment.platform-
ear.ear//org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvoc
ation.java:186)
at deployment.platform-
ear.ear//org.springframework.aop.framework.CglibAopProxy\$CglibMethodInvocation.proceed(CglibAopPro
xy.java:753)
at deployment.platform-
ear.ear//org.springframework.aop.framework.CglibAopProxy\$DynamicAdvisedInterceptor.intercept(CglibA
opProxy.java:698)
at deployment.job-
web.war//com.ideas.tetris.pacman.services.crudservice.TenantCrudServiceBean\$\$EnhancerBySpringCGLIB
\$\$d01ad9d4.flushAndClear(<generated>)
at deployment.platform-ear.ear.ejb-
**pacman.jar//com.ideas.tetris.pacman.services.componentrooms.services.ComponentRoomRevenueSplitS
ervice.splitComponentRoomRevenueForMktAccomActivities(ComponentRoomRevenueSplitService.java:
132)**
at deployment.platform-ear.ear.ejb-
pacman.jar//com.ideas.tetris.pacman.services.componentrooms.services.ComponentRoomRevenueSplitServ
ice.revenueRelatedCorrection(ComponentRoomRevenueSplitService.java:40)
at deployment.job-
web.war//com.ideas.tetris.pacman.services.componentrooms.services.ComponentRoomRevenueSplitServic
e\$\$FastClassBySpringCGLIB\$\$f10f301c.invoke(<generated>)
at deployment.platform-
ear.ear//org.springframework.cglib.proxy.MethodProxy.invoke(MethodProxy.java:218)
at deployment.platform-
ear.ear//org.springframework.aop.framework.CglibAopProxy\$CglibMethodInvocation.invokeJoinpoint(Cglib
AopProxy.java:783)
at deployment.platform-
ear.ear//org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvoc
ation.java:163)
at deployment.platform-
ear.ear//org.springframework.aop.framework.CglibAopProxy\$CglibMethodInvocation.proceed(CglibAopPro
xy.java:753)
at deployment.platform-
ear.ear//org.springframework.transaction.interceptor.TransactionInterceptor\$1.proceedWithInvocation(Tra
nsactionInterceptor.java:123)
at deployment.platform-
ear.ear//org.springframework.transaction.interceptor.TransactionAspectSupport.invokeWithinTransaction(T
ransactionAspectSupport.java:388)
at deployment.platform-
ear.ear//org.springframework.transaction.interceptor.TransactionInterceptor.invoke(TransactionInterceptor.
java:119)
at deployment.platform-
ear.ear//org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvoc
ation.java:186)
at deployment.platform-
ear.ear//org.springframework.aop.framework.CglibAopProxy\$CglibMethodInvocation.proceed(CglibAopPro
xy.java:753)
at deployment.platform-
ear.ear//org.springframework.aop.framework.CglibAopProxy\$DynamicAdvisedInterceptor.intercept(CglibA
opProxy.java:698)
at deployment.job-
web.war//com.ideas.tetris.pacman.services.componentrooms.services.ComponentRoomRevenueSplitServic
e\$\$EnhancerBySpringCGLIB\$\$440608c2.revenueRelatedCorrection(<generated>)
at deployment.job-
web.war//com.ideas.tetris.jems.common.step.componentroom.ComponentRoomRevenueSplitStep.doInvoke
(ComponentRoomRevenueSplitStep.java:45)
at deployment.job-
web.war//com.ideas.tetris.jems.core.step.tasklet.BusinessServiceStep.invoke(BusinessServiceStep.java:179)
at deployment.job-
web.war//com.ideas.tetris.jems.core.step.tasklet.BusinessServiceStep.lambda\$doExecute\$0(BusinessServic
eStep.java:106)
at deployment.platform-
ear.ear//io.micrometer.core.instrument.AbstractTimer.record(AbstractTimer.java:149)
at deployment.job-

```
web.war//com.ideas.tetris.jems.core.step.tasklet.BusinessServiceStep.doExecute(BusinessServiceStep.java:104)
at deployment.job-
web.war//com.ideas.tetris.jems.core.step.tasklet.AbstractTaskletStep.execute(AbstractTaskletStep.java:99)
at deployment.job-
web.war//org.springframework.batch.core.step.tasklet.TaskletStep$ChunkTransactionCallback.doInTransaction(TaskletStep.java:407)
at deployment.job-
web.war//org.springframework.batch.core.step.tasklet.TaskletStep$ChunkTransactionCallback.doInTransaction(TaskletStep.java:331)
at deployment.platform-
ear.ear//org.springframework.transaction.support.TransactionTemplate.execute(TransactionTemplate.java:140)
at deployment.job-
web.war//org.springframework.batch.core.step.tasklet.TaskletStep$2.doInChunkContext(TaskletStep.java:273)
at deployment.job-
web.war//org.springframework.batch.core.scope.context.StepContextRepeatCallback.doInIteration(StepContextRepeatCallback.java:82)
at deployment.job-
web.war//org.springframework.batch.repeat.support.RepeatTemplate.getNextResult(RepeatTemplate.java:375)
at deployment.job-
web.war//org.springframework.batch.repeat.support.RepeatTemplate.executeInternal(RepeatTemplate.java:215)
at deployment.job-
web.war//org.springframework.batch.repeat.support.RepeatTemplate.iterate(RepeatTemplate.java:145)
at deployment.job-
web.war//org.springframework.batch.core.step.tasklet.TaskletStep.doExecute(TaskletStep.java:258)
at deployment.job-
web.war//org.springframework.batch.core.step.AbstractStep.execute(AbstractStep.java:208)
at deployment.job-
web.war//org.springframework.batch.core.job.SimpleStepHandler.handleStep(SimpleStepHandler.java:152)
at deployment.job-
web.war//com.ideas.tetris.jems.core.job.JemsStepHandler.doHandleStep(JemsStepHandler.java:169)
at deployment.job-
web.war//com.ideas.tetris.jems.core.job.JemsStepHandler.handleStepWithRetry(JemsStepHandler.java:113)
at deployment.job-
web.war//com.ideas.tetris.jems.core.job.JemsStepHandler.handleStep(JemsStepHandler.java:76)
at deployment.job-
web.war//org.springframework.batch.core.job.flow.JobFlowExecutor.executeStep(JobFlowExecutor.java:68)
at deployment.job-
web.war//org.springframework.batch.core.job.flow.support.state.StepState.handle(StepState.java:68)
at deployment.job-
web.war//org.springframework.batch.core.job.flow.support.SimpleFlow.resume(SimpleFlow.java:169)
at deployment.job-
web.war//org.springframework.batch.core.job.flow.support.SimpleFlow.start(SimpleFlow.java:144)
at deployment.job-
web.war//com.ideas.tetris.jems.core.job.flow.JemsFlowJob.doExecute(JemsFlowJob.java:36)
at deployment.job-
web.war//org.springframework.batch.core.job.AbstractJob.execute(AbstractJob.java:320)
at deployment.job-
web.war//org.springframework.batch.core.launch.support.SimpleJobLauncher$1.run(SimpleJobLauncher.java:149)
at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1128)
at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:628)
at java.base/java.lang.Thread.run(Thread.java:834)
```

=====

Resolution step followed by Soibam

Soibam simply – restarted the job and resolved the issue .

Revision History

Version	Date	Reasons for Change	Author
1	18-Apr-25	Original publication	Erin & Yogi
1.1	12-May-25	Processing Failure resolution	Yogi
1.2			
1.3			
1.4			
1.5			
1.6			
1.7			

👍 +1

Tagged on: atlantis bahamas paradise

👤 idnyos 📅 24/04/2025 📁 G3 Processes 💬 No Comments

← Core OHIP Installation Process

OHIP/Enable ohipEmulatorUseCorrelationIdOnObtainingDecisions →

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

☐ Save my name, email, and website in this browser for the next time I comment.

Post Comment

